

PRÁCTICA: TRABAJO COLABORATIVO, GESTIÓN DE PULL REQUESTS Y RESOLUCIÓN DE CONFLICTOS EN GITHUB

Nombre: Rodrigo GARCÍA Delgado y Carlos Estévez González

Curso: 2º de Ciclo Superior de Desarrollo de Aplicaciones Web.

ÍNDICE

- Introducción
- Objetivos
- Material empleado
- Desarrollo
- Conclusiones

Introducción.

En el desarrollo de aplicaciones web, el uso de sistemas de control de versiones distribuidos como git y plataformas de alojamiento de código como GitHub es fundamental. El flujo de trabajo profesional se basa en la colaboración multiusuario mediante *forks*, *feature branches*, e hilos de revisión de código conocidos como *Pull Requests*.

Uno de los desafíos más comunes en los equipos de desarrollo es la integración de código concurrente, lo que a menudo genera conflictos de fusión. Estos ocurren cuando dos desarrolladores modifican las mismas líneas de un mismo archivo en diferentes ramas. Git no puede determinar automáticamente cuál es el cambio correcto, por lo que exige una intervención manual para auditar, limpiar y consolidar la versión final del software antes de su despliegue o etiquetado de producción.

Objetivos.

- Establecer roles de trabajo (`user1` como propietario del upstream y `user2` como colaborador externo) simulan un flujo de contribución de código abierto o interdepartamental.
- Dominar el flujo de Pull Requests (PR) mediante la bifurcación de proyectos, la creación de ramas independientes, la revisión remota de código y la retroalimentación directa en el hilo del repositorio.
- Gestionar el ciclo de vida de Issues, vinculando ramas y confirmaciones locales para automatizar el cierre de tareas pendientes en GitHub.
- Resolver conflictos de código de manera manual en local, seleccionando la lógica de negocio adecuada y actualizando el repositorio original de forma segura.

- Versionar el software mediante el uso de etiquetas semánticas y la publicación formal de una *Release* en entornos remotos.

Material empleado.

- Software de control de versiones: Git instalado localmente en las estaciones de trabajo.
- Plataforma Cloud: Dos cuentas activas en GitHub
- Entorno de desarrollo: Editor de código y una terminal.
- Archivos base de la práctica.

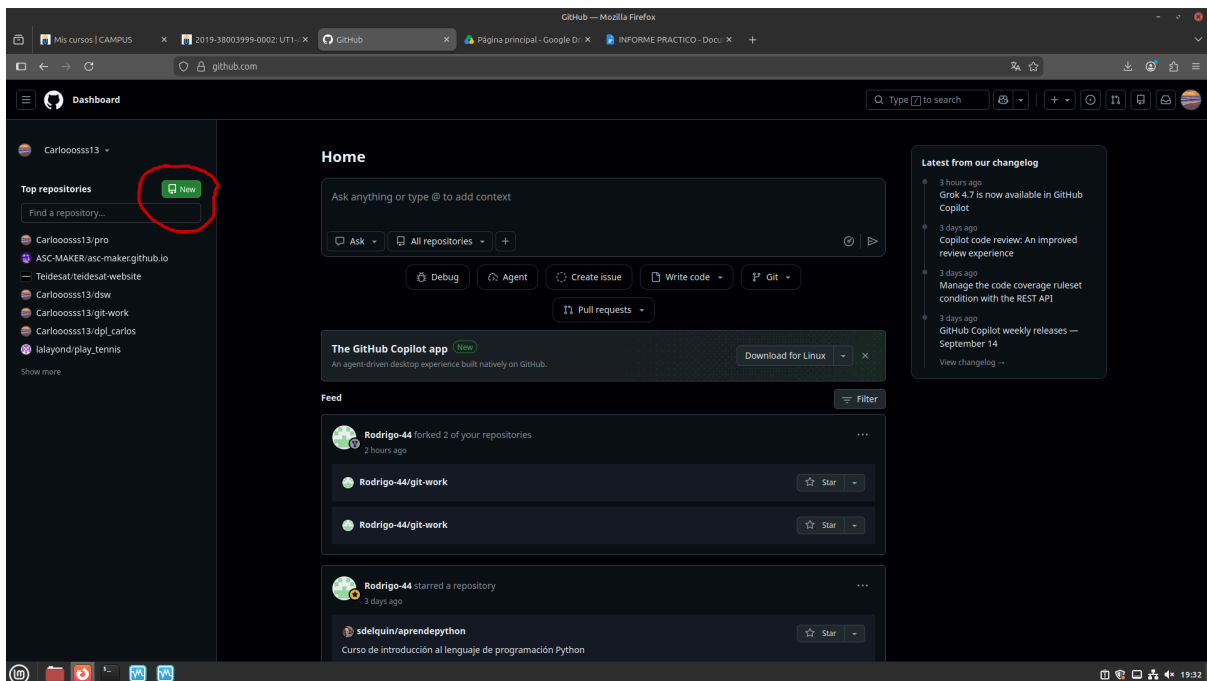
Desarrollo.

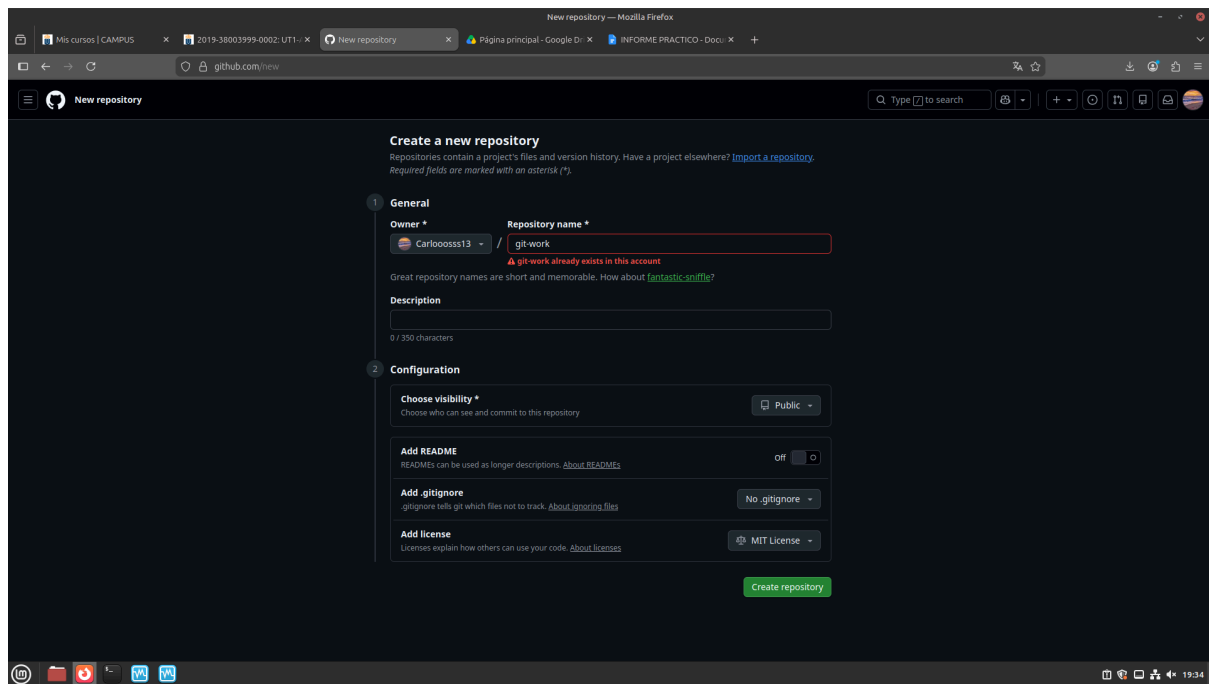
Para la correcta ejecución de esta práctica, se asignan los siguientes identificadores de usuario:

- **User 1**
- **User 2**

1. Inicialización en GitHub (User 1)

User 1 inicia sesión en su cuenta de GitHub y crea un nuevo repositorio público llamado `git-work`. Durante el asistente de configuración, marca obligatoriamente las casillas para incluir un archivo `README.md` y añade una licencia MIT.





2. Descarga y subida de la estructura base (User 1)

En su máquina local, **User 1** abre la terminal y clona el repositorio recién creado. Posteriormente, introduce los archivos requeridos y los envía a la rama principal.

bash

#CLONAR EL REPOSITORIO

git clone <https://github.com/Carloooosss13/git-work.git>

#CREAR LOS ARCHIVOS

touch index.html bootstrap.min.css cover.css

#SUBIMOS LOS CAMBIOS

git add .

git commit -m "Initial project files"

git push origin main

3. Bifurcación del proyecto (User 2)

User 2 accede a la URL pública del repositorio de **User 1** en GitHub y hace clic en el botón **Fork** situado en la esquina superior derecha. Esto genera una copia idéntica del repositorio bajo el espacio de nombres de **User 2**. Acto seguido, lo descarga en su propia máquina local:

bash

user2@maquina-local:~\$ git clone https://github.com

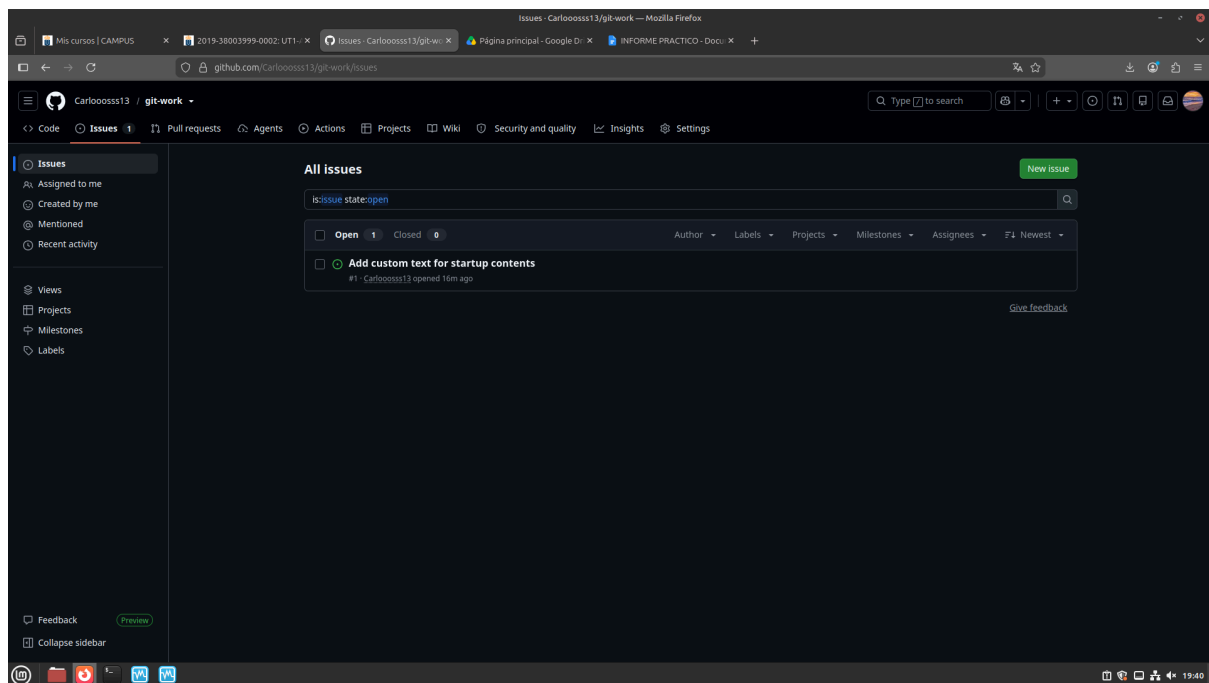
user2@maquina-local:~\$ cd git-work

4. Apertura del primer Issue y desarrollo en rama (User 1 y User 2)

- **User 1** se dirige a la sección *Issues* de su repositorio original y crea una nueva con el título exacto: **Add custom text for startup contents**. El sistema le asigna el identificador **#1**.
- **User 2**, al ver la incidencia abierta, crea una rama local llamada **custom-text** para realizar la tarea, modifica el archivo **index.html** con contenido personalizado para una supuesta startup y sube la rama a su *fork*:

bash

```
git checkout -b custom-text
# [User 2 edita aquí el fichero index.html]
git add index.html
git commit -m "Añadido texto corporativo para la startup"
git push origin custom-text
```



Una vez subido, **User 2** accede a GitHub y abre un **Pull Request (PR)** apuntando desde su rama **USER2:custom-text** hacia la rama **USER1:main**.

5. Revisión remota e inserción de cambios (User 1)

Para probar los cambios en su máquina local antes de aceptarlos, **User 1** añade el repositorio de **User 2** como un origen remoto secundario llamado **upstream**:

```
PS C:\Users\Usuario\Trabajo_Carlos\git-work> git checkout -b custom-text
Switched to a new branch 'custom-text'
PS C:\Users\Usuario\Trabajo_Carlos\git-work> git branch
* custom-text
  main
PS C:\Users\Usuario\Trabajo_Carlos\git-work> |
```

Estando en esa rama, **User 1** realiza cambios adicionales sobre el archivo `index.html` local y los sube **directamente al mismo PR** a través de su enlace remoto:

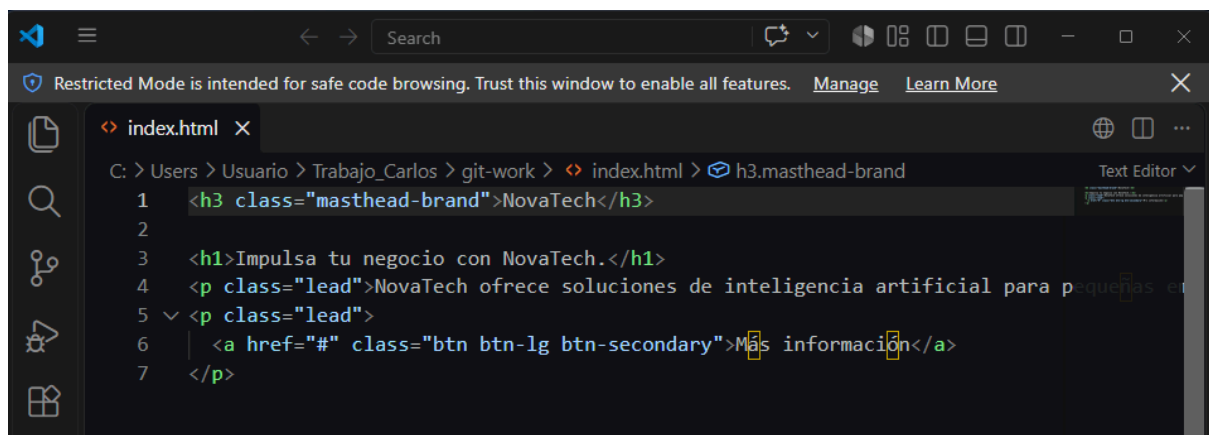
bash

```
git add index.html
git commit -m "User 1: Corrección menor de formato en landing"
git push upstream custom-text
```

6. Conversación en GitHub e integración final

En la interfaz web de GitHub, dentro del hilo del PR, ambos alumnos mantienen una breve conversación técnica.

- Cada usuario realiza **al menos una modificación adicional** en sus respectivos entornos locales sobre la rama `custom-text` y la vuelve a subir para actualizar de forma dinámica la vista del Pull Request.
- Finalmente, **User 1** aprueba el PR en GitHub haciendo clic en **Merge Pull Request**. Para cerrar el issue número 1 de forma limpia, se añade en el mensaje de fusión la sintaxis `Closes #1`.



7. Sincronización de ramas principales

Tras la fusión, ambos alumnos deben actualizar sus ramas principales en local:

- **User 1:**

```
git checkout main
```

```
git pull origin main
```

- **User 2** vincula el repositorio original de User 1 para actualizar su propia copia:

bash

```
git remote add upstream https://github.com
git fetch upstream
git checkout main
git merge upstream/main
git push origin main
```

8. Creación del segundo Issue y commit aislado (User 1)

User 1 abre una segunda issue titulada **Improve UX with cool colors** (identificador #2). A continuación, modifica de forma local el archivo **cover.css**, cambiando la **línea 10** a **color: purple;**. Registra el cambio localmente pero **no realiza el comando git push**:



9. Modificación concurrente y envío de PR (User 2)

Simultáneamente, **User 2** crea una rama llamada **cool-colors**, edita la **línea 10** del mismo archivo **cover.css** asignando el valor **color: darkgreen;** y realiza la subida para solicitar integración:

```
git checkout -b cool-colors
# [User 2 cambia la línea 10 de cover.css a: color: darkgreen;]
git add cover.css
git commit -m "Mejora visual con verde oscuro"
git push origin cool-colors
```

User 2 envía el correspondiente Pull Request a **User 1**. GitHub notificará de inmediato que el archivo posee cambios incompatibles y no se puede fusionar automáticamente.

10. Gestión del conflicto en el entorno local (User 1)

User 1 descarga la rama del conflicto a su entorno local para solventar el bloqueo:

```
git fetch upstream cool-colors
git checkout main
git merge upstream/cool-colors
```

La terminal arrojará el siguiente aviso de error:

```
CONFLICT (content): Merge conflict in cover.css. Automatic merge
failed; fix conflicts and then commit the result.
```

11. Edición del conflicto y cierre de Issue

User 1 abre el archivo `cover.css` con su editor. Se encontrarán los delimitadores estándar de conflicto de Git:

CSS

```
<<<<<<< HEAD
color: purple;
=====
color: darkgreen;
>>>>>>> upstream/cool-colors
```

Siguiendo las instrucciones de la práctica, **User 1** elimina los marcadores y **mantiene el valor propuesto por User 2** (`color: darkgreen;`). Acto seguido, introduce en la **línea 11** la siguiente propiedad: `text-shadow: 2px 2px 8px lightgreen;`.

Para finalizar el proceso, guarda el fichero, realiza el commit vinculando la issue #2 y sube los cambios consolidados a la rama de producción:

bash

```
git add cover.css
git commit -m "Resolución conflicto: Mantenido verde de User 2 y
añadida sombra. Closes #2"
git push origin main
```

12. Creación del Tag y Release en GitHub (User 1)

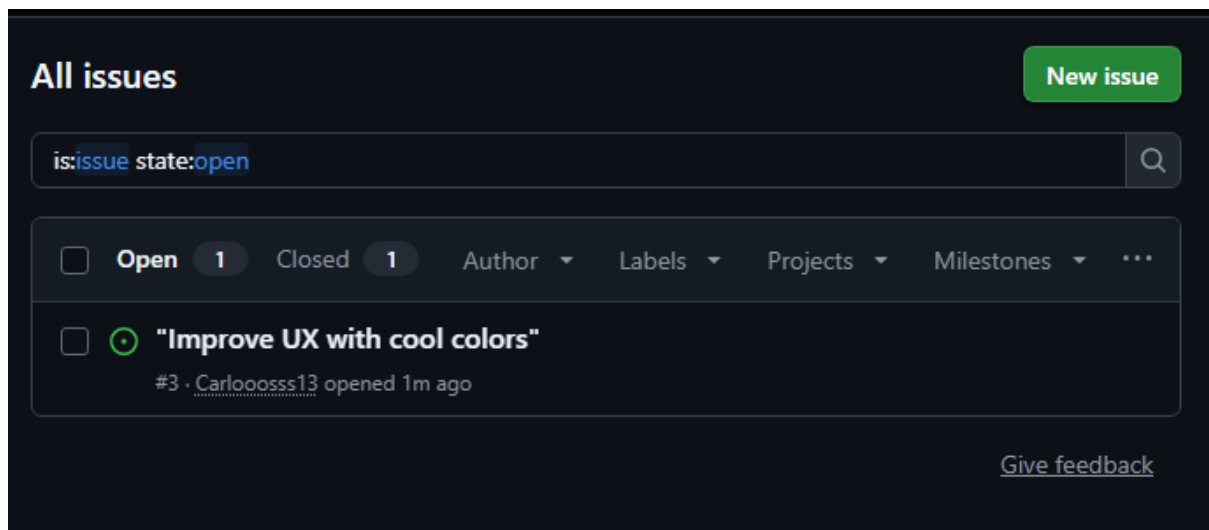
Con las dos características integradas de forma satisfactoria, **User 1** define el estado actual del repositorio como la primera versión estable utilizando etiquetas de Git:

bash

```
git tag -a v0.1.0 -m "Versión 0.1.0 inicial con estilos corregidos y  
sombra de texto"  
git push origin --tags
```

Finalmente, **User 1** accede a su panel del repositorio en GitHub, navega hasta la sección **Releases**, selecciona la etiqueta `v0.1.0` recién subida, introduce el nombre del lanzamiento y pulsa sobre **Publish Release**. El archivo queda empaquetado y listo para su distribución.

A continuación dejamos una serie de capturas desde el segundo issue hacia delante del trabajo realizado.



```
mint@mint:~/git-work$ nano cover.css
mint@mint:~/git-work$ git add cover.css
mint@mint:~/git-work$ git commit -m "Change color to purple in cover.css"
[main 9736b2a] Change color to purple in cover.css
1 file changed, 1 insertion(+), 1 deletion(-)
mint@mint:~/git-work$
```

```
/*
 * Globals
 */

/* Custom default button */
.btn-secondary,
.btn-secondary:hover,
.btn-secondary:focus {
  color: purple;
  text-shadow: none; /* Prevent inheritance from `body` */
}

/*
 * Base structure
 */
```

```
PS C:\Users\Usuario\Trabajo_Carlos\git-work> git branch
* cool-colors
  custom-text
  main
```

```

/*
 * Globals
 */

/* Custom default button */
.btn-secondary,
.btn-secondary:hover,
.btn-secondary:focus {
  color: ■darkgreen;
  text-shadow: none; /* Prevent inheritance from `body` */
}

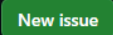
```

```

PS C:\Users\Usuario\Trabajo_Carlos\git-work> git add cover.css
PS C:\Users\Usuario\Trabajo_Carlos\git-work> git commit -m "Change color to darkgreen in cover.css"
[cool-colors 939e110] Change color to darkgreen in cover.css
1 file changed, 53 insertions(+)
PS C:\Users\Usuario\Trabajo_Carlos\git-work> git push origin cool-colors
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 12 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 651 bytes | 651.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'cool-colors' on GitHub by visiting:
remote:   https://github.com/Rodrigo-44/git-work/pull/new/cool-colors
remote:
To https://github.com/Rodrigo-44/git-work.git
 * [new branch]      cool-colors -> cool-colors
PS C:\Users\Usuario\Trabajo_Carlos\git-work> |

```

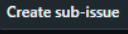
"Improve UX with cool colors" #3

 Closed

 Carloooosss13 opened 1h ago Owner 

No description provided.

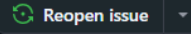
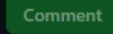
  Carloooosss13 closed this as completed 1m ago

Add a comment

Write Preview H B I          

Use Markdown to format your comment

 Paste, drop, or click to add files

Assignees 
No one - [Assign yourself](#)

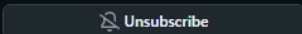
Labels 
No labels

Projects 
No projects

Milestone 
No milestone

Relationships 
None yet

Development 
[Create a branch](#) for this issue or link a pull request.

Notifications Customize


You're receiving notifications because you're subscribed to this thread.

```
mint@mint:~/git-work$ git push origin main
Username for 'https://github.com': Carloooosss13
Password for 'https://Carloooosss13@github.com':
Enumerating objects: 20, done.
Counting objects: 100% (19/19), done.
Delta compression using up to 4 threads
Compressing objects: 100% (14/14), done.
Writing objects: 100% (14/14), 1.46 KiB | 500.00 KiB/s, done.
Total 14 (delta 9), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (9/9), completed with 3 local objects.
To https://github.com/Carloooosss13/git-work.git
   a8a09c5..e2fe830  main -> main
mint@mint:~/git-work$
```

```
mint@mint:~/git-work$ git tag -a 0.1.0 -m "Version 0.1.0"
mint@mint:~/git-work$ git push origin 0.1.0
Username for 'https://github.com': Carloooosss13
Password for 'https://Carloooosss13@github.com':
Enumerating objects: 1, done.
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 167 bytes | 167.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/Carloooosss13/git-work.git
 * [new tag]           0.1.0 -> 0.1.0
mint@mint:~/git-work$
```